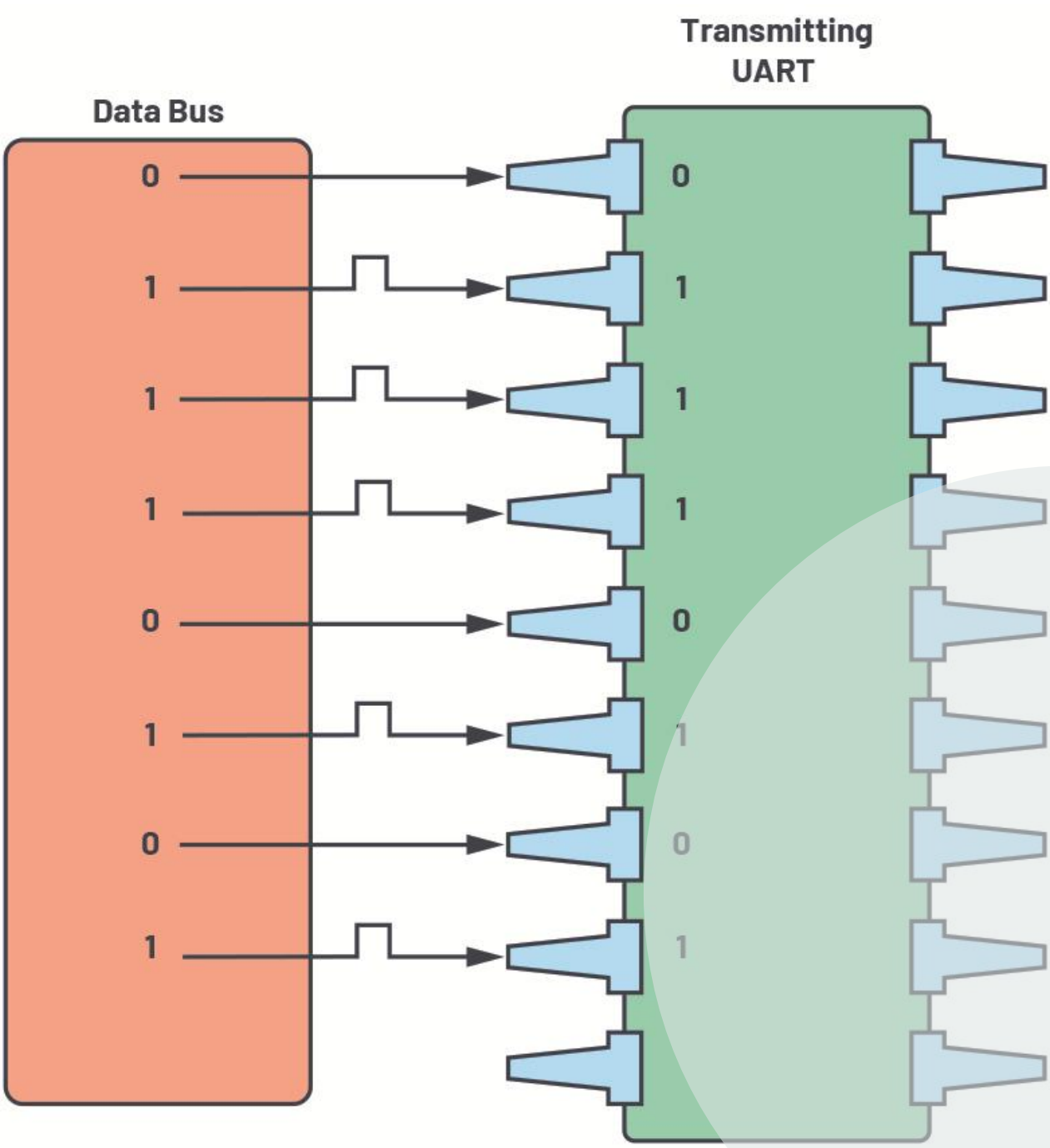


UART_TX

Prepared by:

Arwa Ashraf Kantoush

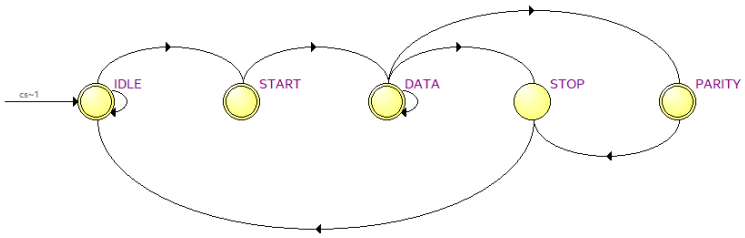
TABLE OF CONTENTS

1. INTRODUCTION & OVERVIEW

1.1 Project Overview & Objectives

The Universal Asynchronous Receiver-Transmitter (UART) is a fundamental protocol for serial communication in digital and embedded systems. This project focuses on designing and verifying a parameterized **UART Transmitter (UART TX)** subsystem using synthesizable **Verilog HDL** to serialize parallel data with high reliability and timing accuracy.

1.2 UART_TX Architecture Summary



	Source State	Destination State	Condition
1	DATA	STOP	(!P_EN).(SER_DONE)
2	DATA	PARITY	(P_EN).(SER_DONE)
3	DATA	DATA	(!SER_DONE)
4	IDLE	IDLE	(!V_INPUT)
5	IDLE	START	(V_INPUT)
6	PARITY	STOP	
7	START	DATA	
8	STOP	IDLE	

The design comprises four core functional blocks:

- **Main Controller (FSM):** Controls transmission states (IDLE, START, DATA, PARITY, STOP) and asserts the BUSY status signal.
- **Serializer:** Converts the N-bit parallel bus into a serial bitstream and generates a completion flag (SER_DONE).
- **Parity Bit Calculator:** Generates the parity bit using reduction XOR/XNOR logic according to user configuration.
- **Multiplexer (MUX):** Directs Start, Data, Parity, or Stop/IDLE bits onto the TX_OUTPUT line.

1.3 Operational Features & Frame Transmission Sequence

- **Flexible Parity:** Supports optional parity (P_EN) with selectable Even/Odd modes (P_BIT).
- **Asynchronous Reset:** Active-low reset (RST) to instantly initialize all internal registers.
- **Single-Cycle Handshake:** Samples input data on a 1-clock-cycle pulse on V_INPUT.
- **Busy Protection:** Asserts BUSY = 1 during transmission and locks out any new data requests until the active frame finishes.
- **Idle Stability:** Maintains TX_OUTPUT = 1 during idle states to prevent false start triggers.

2. RTL DESIGN & MODELING

2.1 Parity Calculator

```
module Parity_Calculator #(
    parameter WIDTH = 8
) (
    input CLK,
    input RST,
    input [WIDTH-1:0]P_INPUT,
    input V_INPUT,
    input P_EN,
    input P_BIT,
    output reg PARITY_BIT
);

always @(posedge CLK or negedge RST) begin
    if (!RST) begin
        PARITY_BIT <= 0;
    end
    else begin
        if (V_INPUT && P_EN) begin
            if (P_BIT) begin
                PARITY_BIT <= ~^P_INPUT;
            end
            else begin
                PARITY_BIT <= ^P_INPUT;
            end
        end
    end
end

endmodule //Parity_Calculator
```

2.2 Serializer

```
module Serializer #(
    parameter WIDTH = 8
) (
    input CLK,
    input RST,
    input [WIDTH-1:0]P_INPUT,
    input V_INPUT,
    input SER_EN,
    output SER_DATA,
    output reg SER_DONE
);
reg [WIDTH-1:0]register;
reg [$clog2(WIDTH)-1:0]counter;

always @(posedge CLK or negedge RST) begin
    if (!RST) begin
        SER_DONE <= 0;
        register <= 0;
        counter <= 0;
    end
    else if (V_INPUT && !SER_EN) begin
            register <= P_INPUT;
            SER_DONE <= 0;
            counter <= 0;
        end
    else if (SER_EN) begin
        if (counter < WIDTH-2) begin
            register <= {1'b0,register[WIDTH-1:1]};
            SER_DONE <= 0;
            counter <= counter + 1;
        end
        else begin
            register <= {1'b0,register[WIDTH-1:1]};
            SER_DONE <= 1;
            counter <= 0;
        end
    end
    else begin
        SER_DONE <= 0;
    end
end

assign SER_DATA = (!RST)? 0 : register[0];

endmodule //Serializer
```

2.3 MUX

```
module MUX #(
    parameter START_BIT = 0,
    parameter STOP_BIT = 1
) (
    input SER_DATA,
    input PARITY_BIT,
    input [1:0]SEL,
    output reg TX_OUTPUT
);

always @(*) begin
    case (SEL)
        2'b00 : begin
            TX_OUTPUT = START_BIT;
        end
        2'b01 : begin
            TX_OUTPUT = SER_DATA;
        end
        2'b10 : begin
            TX_OUTPUT = PARITY_BIT;
        end
        2'b11 : begin
            TX_OUTPUT = STOP_BIT;
        end
        default : begin
            TX_OUTPUT = STOP_BIT;
        end
    endcase
end

endmodule //MUX
```

2.4 FSM

```
module FSM (
    input CLK,
    input RST,
    input V_INPUT,
    input P_EN,
    input SER_DONE,
    output reg SER_EN,
    output reg BUSY,
    output reg [1:0]SEL
);
localparam IDLE = 3'b000;
localparam START = 3'b001;
localparam DATA = 3'b010;
localparam PARITY = 3'b011;
localparam STOP = 3'b100;

reg [2:0]cs,ns;

always @(*) begin
    case (cs)
        IDLE : begin
            if (V_INPUT) begin
                ns = START;
            end
            else begin
                ns = IDLE;
            end
        end
        START : begin
            ns = DATA;
        end
        DATA : begin
            if (SER_DONE) begin
                if (P_EN) begin
                    ns = PARITY;
                end
                else begin
                    ns = STOP;
                end
            end
            else begin
                ns = DATA;
            end
        end
        PARITY : begin
```

```
            ns = STOP;
        end
        STOP : begin
            ns = IDLE;
        end
        default : begin
            ns = IDLE;
        end
    endcase
end

always @(posedge CLK or negedge RST) begin
    if (!RST) begin
        cs <= IDLE;
    end
    else begin
        cs <= ns;
    end
end

always @(*) begin
    case (cs)
        IDLE : begin
            SER_EN = 0;
            BUSY = 0;
            SEL = 2'b11;
        end
        START : begin
            SER_EN = 0;
            BUSY = 1;
            SEL = 2'b00;
        end
        DATA : begin
            SER_EN = 1;
            BUSY = 1;
            SEL = 2'b01;
        end
        PARITY : begin
            SER_EN = 0;
            BUSY = 1;
            SEL = 2'b10;
        end
        STOP : begin
            SER_EN = 0;
            BUSY = 1;
            SEL = 2'b11;
        end
```

```
        default : begin
            SER_EN = 0;
            BUSY = 0;
            SEL = 2'b11;
        end
    endcase
end

endmodule //FSM
```

2.5 TOP-LEVEL MODULE

```
module UART_TX #(
    parameter WIDTH = 8
) (
    input CLK,
    input RST,
    input [WIDTH-1:0]P_INPUT,
    input V_INPUT,
    input P_EN,
    input P_BIT,
    output TX_OUTPUT,
    output BUSY
);
wire PARITY_BIT;
wire SER_EN;
wire SER_DATA;
wire SER_DONE;
wire [1:0]SEL;

Parity_Calculator #(.WIDTH(WIDTH)) dut1 (
    .CLK(CLK),
    .RST(RST),
    .P_INPUT(P_INPUT),
    .V_INPUT(V_INPUT),
    .P_EN(P_EN),
    .P_BIT(P_BIT),
    .PARITY_BIT(PARITY_BIT)
);

Serializer #(.WIDTH(WIDTH)) dut2 (
    .CLK(CLK),
    .RST(RST),
    .P_INPUT(P_INPUT),
    .V_INPUT(V_INPUT),
    .SER_EN(SER_EN),
    .SER_DATA(SER_DATA),
    .SER_DONE(SER_DONE)
);

MUX dut3 (
    .SER_DATA(SER_DATA),
    .PARITY_BIT(PARITY_BIT),
    .SEL(SEL),
    .TX_OUTPUT(TX_OUTPUT)
);
```

```
FSM dut (
    .CLK(CLK),
    .RST(RST),
    .V_INPUT(V_INPUT),
    .P_EN(P_EN),
    .SER_DONE(SER_DONE),
    .SER_EN(SER_EN),
    .BUSY(BUSY),
    .SEL(SEL)
);

endmodule //TOP
```

3. FUNCTIONAL VERIFICATION & TESTBENCH

```
module UART_TX_tb ();
parameter WIDTH = 8;
reg CLK;
reg RST;
reg [WIDTH-1:0]P_INPUT;
reg V_INPUT;
reg P_EN;
reg P_BIT;
wire TX_OUTPUT;
reg TX_OUTPUT_exp;
wire BUSY;
reg BUSY_exp;

UART_TX #(.WIDTH(WIDTH)
    ) dut ( .CLK(CLK),
            .RST(RST),
            .P_INPUT(P_INPUT),
            .V_INPUT(V_INPUT),
            .P_EN(P_EN),
            .P_BIT(P_BIT),
            .TX_OUTPUT(TX_OUTPUT),
            .BUSY(BUSY)
);

initial begin
    CLK = 1;
    forever begin
        #1 CLK = ~CLK;
    end
end

integer i;
initial begin
    RST = 0;
    P_INPUT = 8'b0000_0000; V_INPUT = 0; P_EN = 0; P_BIT = 0;
    TX_OUTPUT_exp = 1; BUSY_exp = 0;
    @(negedge CLK);
    if (TX_OUTPUT !== TX_OUTPUT_exp || BUSY !== BUSY_exp)
begin
        $display("ERROR!");
        //$stop;
    end
```

```
    RST = 1;
    P_INPUT = 8'b1010_0101; V_INPUT = 1; P_EN = 0; P_BIT = 0;
    for (i = 0 ; i < 10 ; i = i + 1) begin
        if (i==0) begin
            TX_OUTPUT_exp = 0; BUSY_exp = 1;
        end
        else if (i<9) begin
            TX_OUTPUT_exp = P_INPUT[i-1]; BUSY_exp = 1;
        end
        else begin
            TX_OUTPUT_exp = 1; BUSY_exp = 1;
        end
        @(negedge CLK);
        V_INPUT = 0;
        if (TX_OUTPUT !== TX_OUTPUT_exp || BUSY !== BUSY_exp)
begin
            $display("ERROR!");
            //$stop;
        end
    end

    TX_OUTPUT_exp = 1; BUSY_exp = 0;
    @(negedge CLK);
    if (TX_OUTPUT !== TX_OUTPUT_exp || BUSY !== BUSY_exp)
begin
        $display("ERROR!");
        //$stop;
    end

    P_INPUT = 8'b1010_0101; V_INPUT = 1; P_EN = 1; P_BIT = 0;
    for (i = 0 ; i < 11 ; i = i + 1) begin
        if (i==0) begin
            TX_OUTPUT_exp = 0; BUSY_exp = 1;
        end
        else if (i<9) begin
            TX_OUTPUT_exp = P_INPUT[i-1]; BUSY_exp = 1;
        end
        else if (i<10) begin
            TX_OUTPUT_exp = ^P_INPUT; BUSY_exp = 1;
        end
        else begin
            TX_OUTPUT_exp = 1; BUSY_exp = 1;
        end
        @(negedge CLK);
        V_INPUT = 0;
        if (TX_OUTPUT !== TX_OUTPUT_exp || BUSY !== BUSY_exp)
begin
```

```
            $display("ERROR!");
            //$stop;
        end
    end

    TX_OUTPUT_exp = 1; BUSY_exp = 0;
    @(negedge CLK);
    if (TX_OUTPUT !== TX_OUTPUT_exp || BUSY !== BUSY_exp)
begin
        $display("ERROR!");
        //$stop;
    end

    P_INPUT = 8'b1010_0101; V_INPUT = 1; P_EN = 1; P_BIT = 1;
    for (i = 0 ; i < 11 ; i = i + 1) begin
        if (i==0) begin
            TX_OUTPUT_exp = 0; BUSY_exp = 1;
        end
        else if (i<9) begin
            TX_OUTPUT_exp = P_INPUT[i-1]; BUSY_exp = 1;
        end
        else if (i<10) begin
            TX_OUTPUT_exp = ~^P_INPUT; BUSY_exp = 1;
        end
        else begin
            TX_OUTPUT_exp = 1; BUSY_exp = 1;
        end
        @(negedge CLK);
        V_INPUT = 0;
        if (TX_OUTPUT !== TX_OUTPUT_exp || BUSY !== BUSY_exp)
begin
            $display("ERROR!");
            //$stop;
        end
    end

    TX_OUTPUT_exp = 1; BUSY_exp = 0;
    @(negedge CLK);
    if (TX_OUTPUT !== TX_OUTPUT_exp || BUSY !== BUSY_exp)
begin
        $display("ERROR!");
        //$stop;
    end
    $stop;
end

initial begin
```

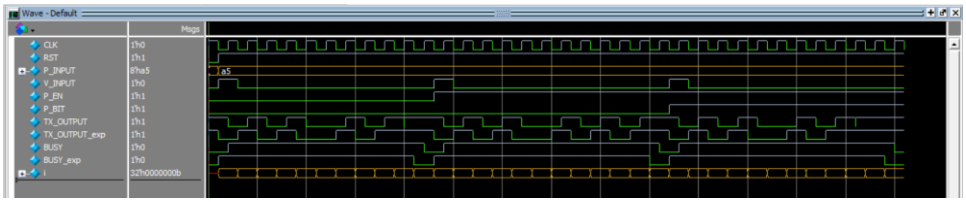
```
    $monitor("CLK=%b,RST=%b,P_INPUT=%b,V_INPUT=%b,P_EN=%b,P_BI
T=%b,TX_OUTPUT=%b,TX_OUTPUT_exp=%b,BUSY=%b,BUSY_exp=%b",
                CLK,RST,P_INPUT,V_INPUT,P_EN,P_BIT,TX_OUTPUT,T
X_OUTPUT_exp,BUSY,BUSY_exp);
end

endmodule //UART_TX_tb
```

4. SIMULATION & AUTOMATION

4.1 QuestaSim Simulation Script (run.do)

```
vlib work
vlog Parity_Calculator.v Serializer.v FSM.v MUX.v UART_TX.v
UART_TX_tb.v
vsim -voptargs=+acc work.UART_TX_tb
add wave *
run -all
#quit -sim
```

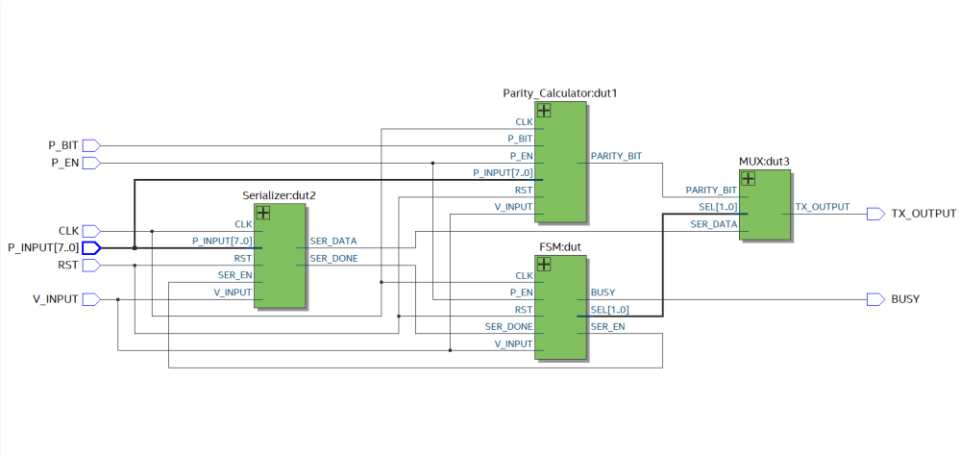
4.2 Simulation Waveforms

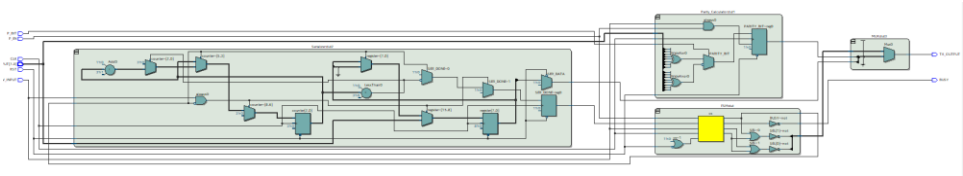


5. DESIGN CONSTRAINTS

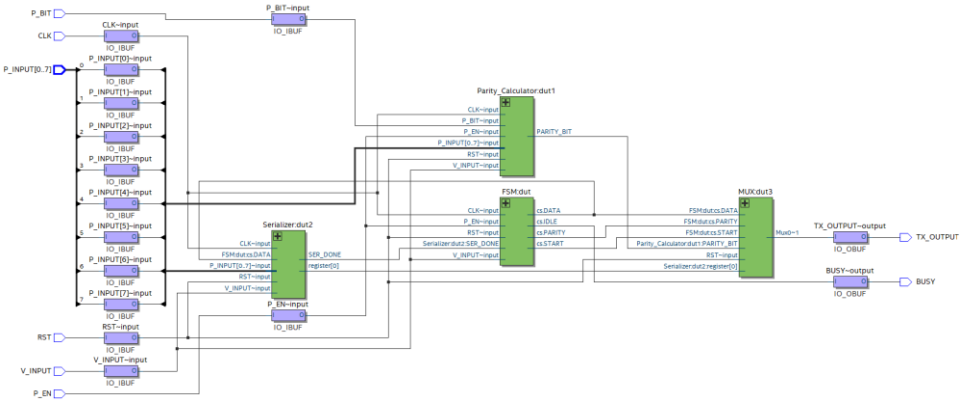
```
# ---- Clock definition ----
create_clock -name CLK -period 20.000 [get_ports CLK]

# ---- Reset ----
set_false_path -from [get_ports RST]

# ---- Input delays ----
set_input_delay -clock CLK -max 3.000 [get_ports {P_INPUT[*]}]
set_input_delay -clock CLK -min 0.500 [get_ports {P_INPUT[*]}]
set_input_delay -clock CLK -max 3.000 [get_ports V_INPUT]
set_input_delay -clock CLK -min 0.500 [get_ports V_INPUT]
set_input_delay -clock CLK -max 3.000 [get_ports {P_EN P_BIT}]
set_input_delay -clock CLK -min 0.500 [get_ports {P_EN P_BIT}]

# ---- Output delays ----
set_output_delay -clock CLK -max 2.000 [get_ports TX_OUTPUT]
set_output_delay -clock CLK -min 0.500 [get_ports TX_OUTPUT]
set_output_delay -clock CLK -max 2.000 [get_ports BUSY]
set_output_delay -clock CLK -min 0.500 [get_ports BUSY]
```

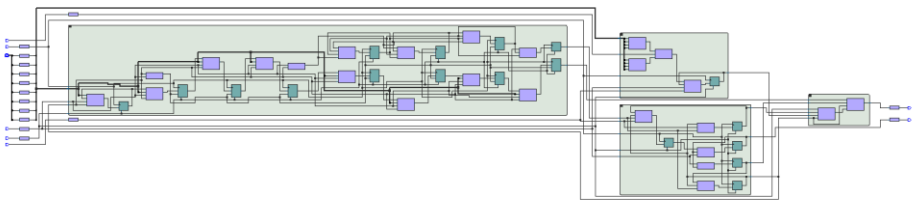
6. ELABORATION FLOW



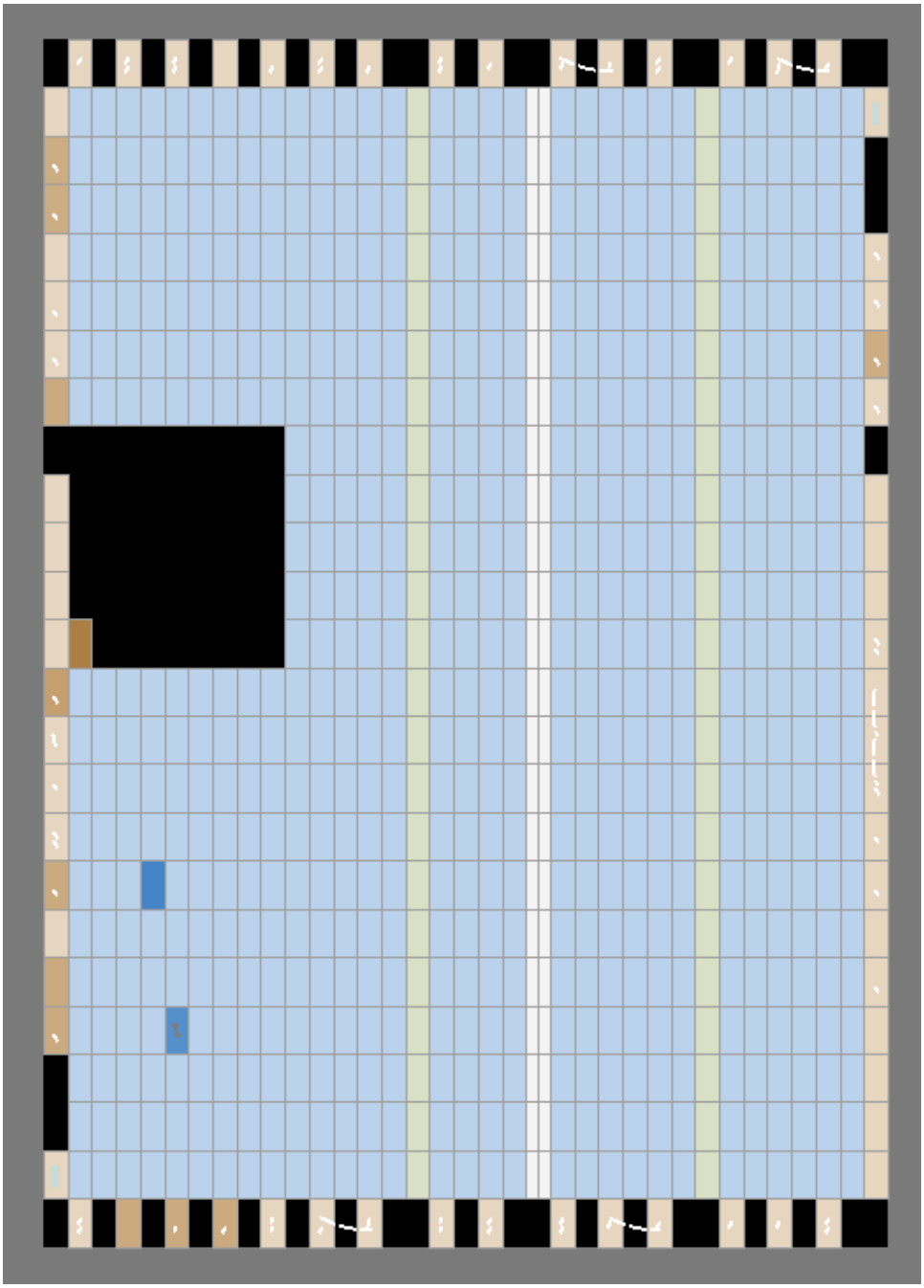


7. SYNTHESIS FLOW





8. IMPLEMENTATION FLOW (PLACE & ROUTE)

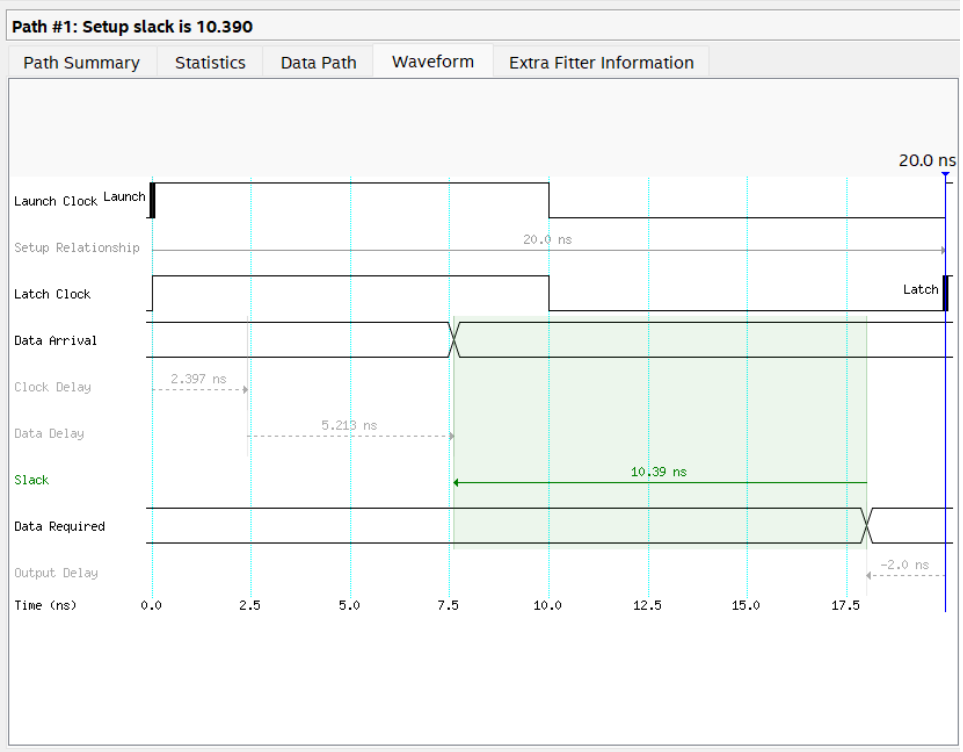


9. TIMING REPORT

Setup Times

	Data Port	Clock Port	Rise	Fall	Clock Edge	Clock Reference
١	P_BIT	CLK	2.946	3.322	Rise	CLK
٢	P_EN	CLK	2.069	2.418	Rise	CLK
٣	P_INPUT[*]	CLK	4.149	4.480	Rise	CLK
١	P_INPUT[0]	CLK	3.547	3.904	Rise	CLK
٢	P_INPUT[1]	CLK	3.573	3.938	Rise	CLK
٣	P_INPUT[2]	CLK	3.530	3.882	Rise	CLK
٤	P_INPUT[3]	CLK	3.446	3.812	Rise	CLK
٥	P_INPUT[4]	CLK	3.655	4.022	Rise	CLK
٦	P_INPUT[5]	CLK	4.149	4.480	Rise	CLK
٧	P_INPUT[6]	CLK	3.750	4.133	Rise	CLK
٨	P_INPUT[7]	CLK	3.759	4.124	Rise	CLK
٤	V_INPUT	CLK	3.917	4.226	Rise	CLK

9.1 Setup slack



Slow 1200mV 125C Model

Command Info | Summary of Paths

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	10.390	Parity_Calculator:dut1|PARITY_BIT	TX_OUTPUT	CLK	CLK	20.000	-2.397	5.213
2	10.395	FSM:dut|cs.PARITY	TX_OUTPUT	CLK	CLK	20.000	-2.397	5.208
3	10.513	Serializer:dut2|register[0]	TX_OUTPUT	CLK	CLK	20.000	-2.399	5.088
4	10.657	FSM:dut|cs.DATA	TX_OUTPUT	CLK	CLK	20.000	-2.397	4.946
5	10.868	FSM:dut|cs.START	TX_OUTPUT	CLK	CLK	20.000	-2.397	4.735
6	11.148	FSM:dut|cs.DATA	TX_OUTPUT	CLK	CLK	20.000	-2.397	4.455
7	11.626	FSM:dut|cs.IDLE	BUSY	CLK	CLK	20.000	-2.397	3.977
8	12.520	P_INPUT[5]	Parity...TY_BIT	CLK	CLK	20.000	2.307	6.804
9	12.774	V_INPUT	Seriali...nter[1]	CLK	CLK	20.000	2.307	6.550
10	12.774	V_INPUT	Seriali...nter[0]	CLK	CLK	20.000	2.307	6.550

Path #1: Setup slack is 10.390

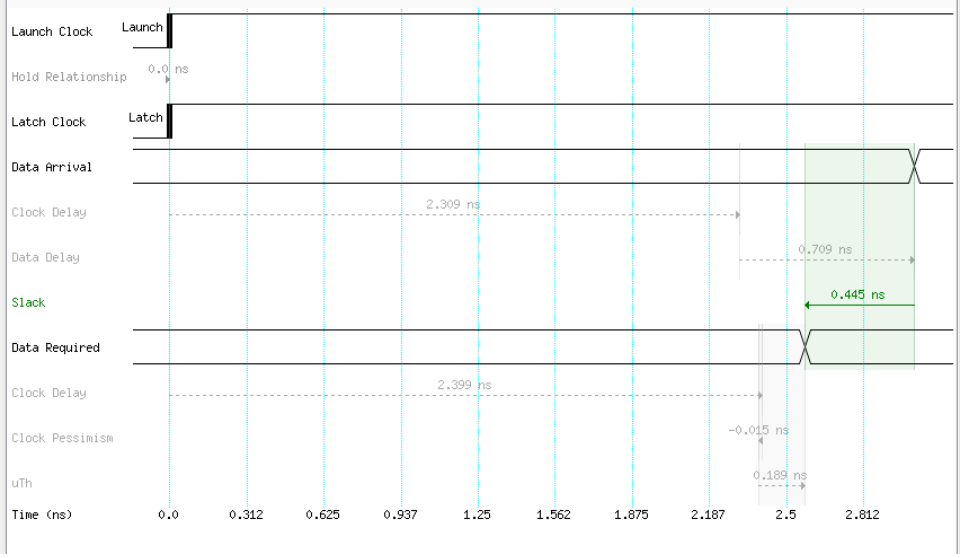
Path Summary | Statistics | Data Path | Waveform | Extra Fitter Information

	Property	Value
1	From Node	Parity_Calculator:dut1|PARITY_BIT
2	To Node	TX_OUTPUT
3	Launch Clock	CLK
4	Latch Clock	CLK
5	Data Arrival Time	7.610
6	Data Required Time	18.000
7	Slack	10.390

9.2Hold slack

Path #10: Hold slack is 0.445

Path Summary | Statistics | Data Path | Waveform | Extra Fitter Information



Slow 1200mV 125C Model

Command Info | Summary of Paths

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
١	0.428	Parity_Calculator:dut1|PARITY_BIT	Parity...TY_BIT	CLK	CLK	0.000	0.075	0.692
٢	0.428	Serializer:dut2|register[7]	Seriali...ster[7]	CLK	CLK	0.000	0.075	0.692
٣	0.428	FSM:dut|cs.DATA	FSM:d....DATA	CLK	CLK	0.000	0.075	0.692
٤	0.428	Serializer:dut2|counter[1]	Seriali...nter[1]	CLK	CLK	0.000	0.075	0.692
٥	0.428	Serializer:dut2|counter[2]	Seriali...nter[2]	CLK	CLK	0.000	0.075	0.692
٦	0.428	FSM:dut|cs.IDLE	FSM:d....IDLE	CLK	CLK	0.000	0.075	0.692
٧	0.429	Serializer:dut2|counter[0]	Seriali...nter[0]	CLK	CLK	0.000	0.075	0.693
٨	0.442	FSM:dut|cs.STOP	FSM:d....IDLE	CLK	CLK	0.000	0.075	0.706
٩	0.443	Serializer:dut2|register[6]	Seriali...ster[5]	CLK	CLK	0.000	0.075	0.707
١٠	0.445	Serializer:dut2|register[1]	Seriali...ster[0]	CLK	CLK	0.000	0.075	0.709

Path #10: Hold slack is 0.445

Path Summary | Statistics | Data Path | Waveform | Extra Fitter Information

	Property	Value
١	From Node	Serializer:dut2|register[1]
٢	To Node	Serializer:dut2|register[0]
٣	Launch Clock	CLK
٤	Latch Clock	CLK
٥	Data Arrival Time	3.018
٦	Data Required Time	2.573
٧	Slack	0.445

10. LINTING